Digital VLSI Design

Assignment-1

4-bit Array Multiplier from RTL to GDSII with Open-Source Tools

Sai Rithvik Achutuni Gautam Gandhi

Abstract

In this assignment we implemented a 4-bit unsigned array multiplier in Verilog, verified it, and took it through a complete open-source RTL-to-GDSII flow on the SkyWater 130nm process.
This assignment is split up into 4 parts, with each part covering as follows:

- **Part-A**: RTL Design and Verification
- **Part-B**: Logic Synthesis with Yosys
- **Part-C**: Static Timing Analysis with OpenSTA
- **Part-D**: Physical Design

In this report we seek to go through in detail about how we implemented all these parts and then report the deliverables. At the end, the github repo link and individual contributions are listed.

Introduction

1 Part-A: RTL Design and Verification

In this part, we implemented three files:

- `rtl/full_adder.v`
- `rtl/mult_array.v`
- `rtl/tb_mult.v`

1.1 Tasks, Implementation Details

In this part, we implement RTL-design and then verified it. The functions of each files are:

- `full_adder.v`: single-bit full adder
- `mult_array.v`: the 4×4 array multiplier itself

- `tb_mult.v`: self checking testbench. Drives a 10ns clock and prints one PASS/FAIL summary line (checks all 256 (a, b) combinations)

`mult_array.v` registers a/b on the clock edge, forms 4 partial products, adds them through 3 rows of 4 full adders (ripple-carry), and registers the 8-bit result into p – 16 flip-flops, no `*` operator, 2-cycle latency. Simulated with `iverilog` + `vvp`, waveform viewed in `gtkwave`.

1.2 Deliverables

Chose Roll number: 2023102060. So $A = 5, B = 7$ this implies, $P = A \times B = 35$. The waveform screenshot is below:

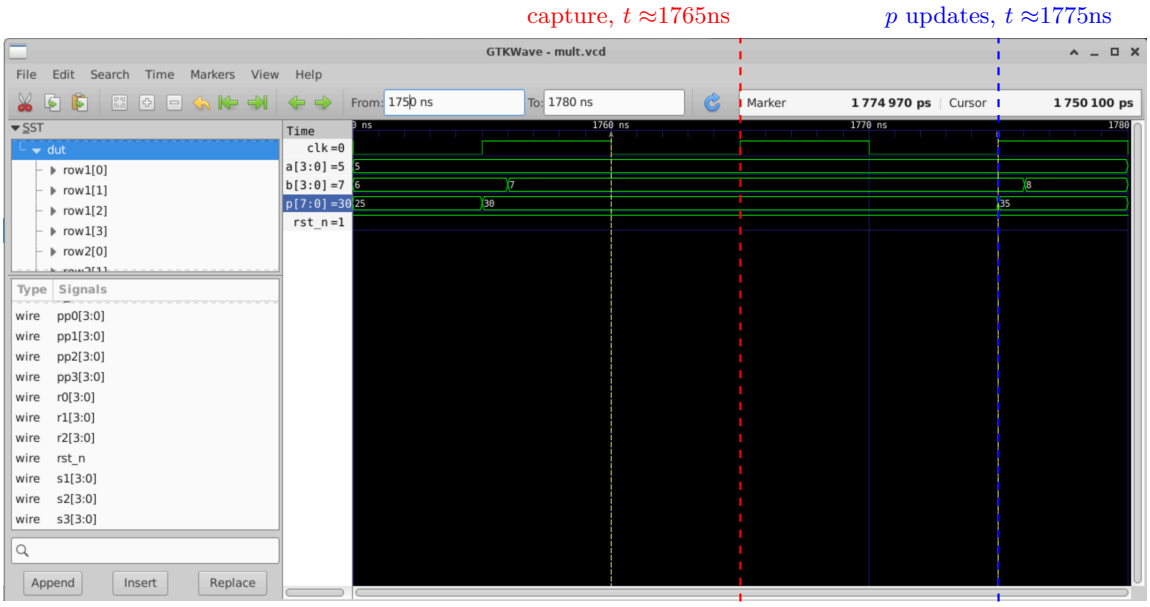


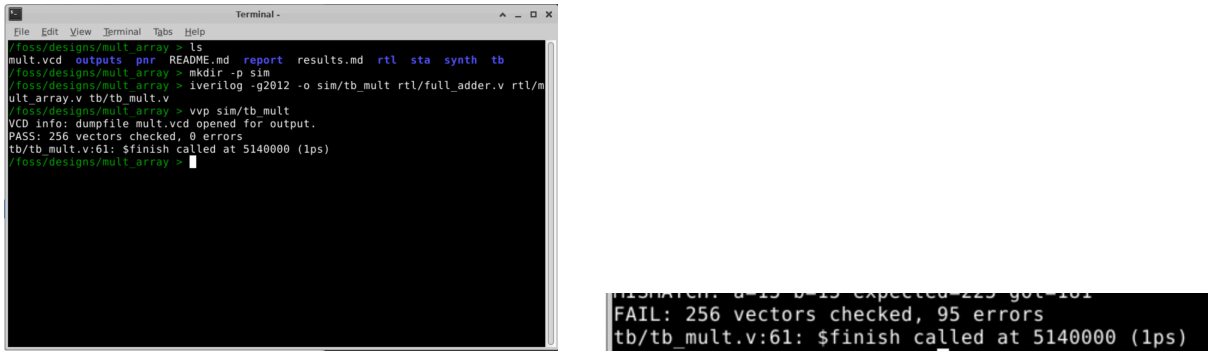
Figure 1. Waveform with the 2-cycle latency marked. Red: a, b captured into a_reg, b_reg ($t \approx$ 1765ns). Blue: p updates to the final product ($t \approx$ 1775ns) – one clock period (10ns) later.

1.3 Hand-worked Example

Using roll number 2023102060 (last digit 0): $A = 5$, $B = 7$.

```
        0 1 0 1   A (5)
      x 0 1 1 1   B (7)
      ---------
        0 1 0 1   pp0 = A AND b0 (b0=1)
      0 1 0 1     pp1 = A AND b1 (b1=1), shifted 1
    0 1 0 1       pp2 = A AND b2 (b2=1), shifted 2
  0 0 0 0         pp3 = A AND b3 (b3=0), shifted 3
---------------
0 0 1 0 0 0 1 1   P = 35
```

Another deliverable is to show the 'PASS/FAIL' commands for the tests. This is to verify if the multiplier is working properly or not. I am pasting the screenshots of the terminal, in the gh repo we have given list of commands that can be tried to recreate the same.



(a) Pass command **(b)** Fail command

Figure 2. Testbench PASS/FAIL output

1.4 Block Diagram

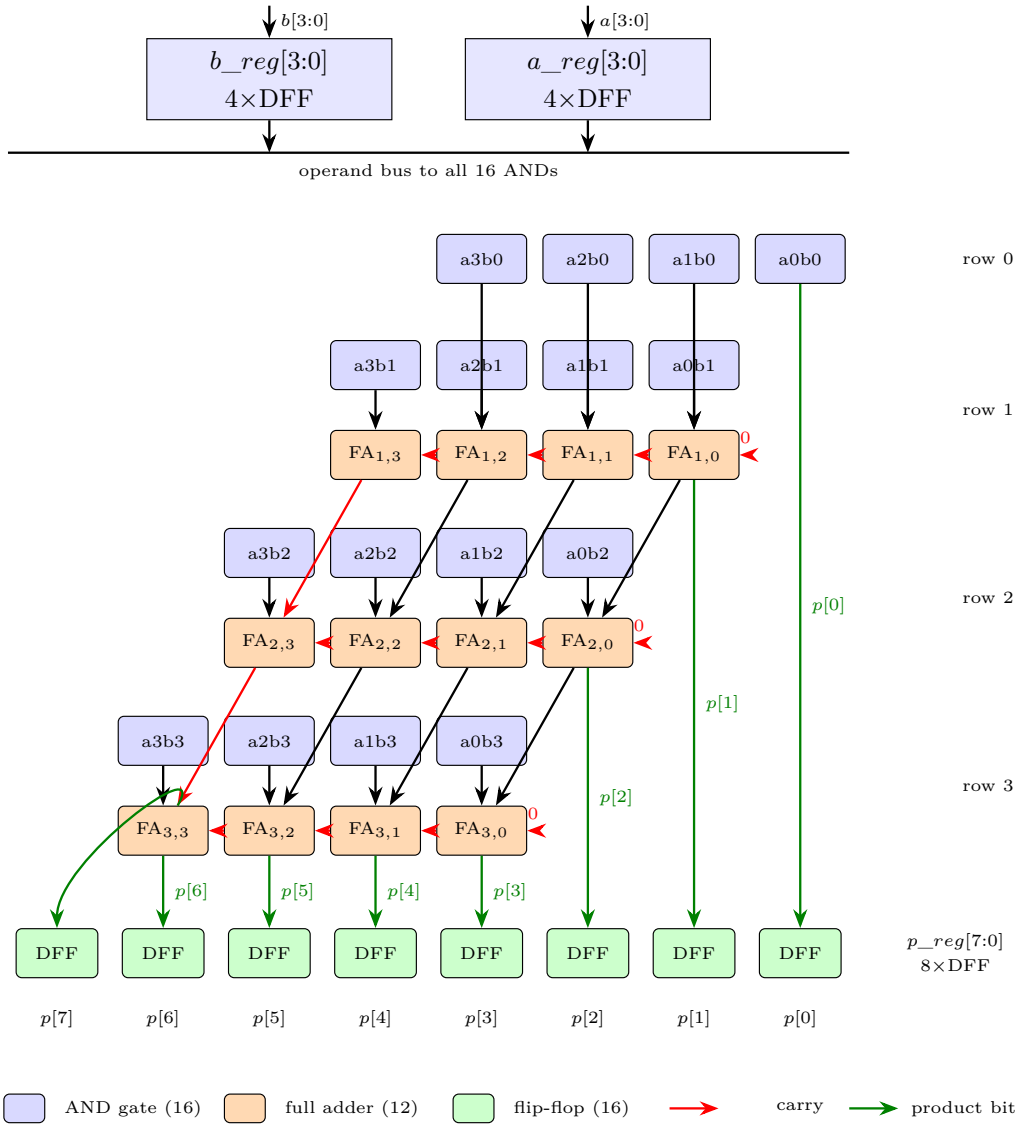


Figure 3. Block diagram of `mult_array`: a/b registered into 4-bit registers; 16 AND gates form partial products $pp0..pp3$; 3 rows of 4 full adders (12 total) sum them with a ripple carry chain (red) per row; each row taps one finished product bit (green) directly into $p_reg[7:0]$.

2 Part-B: Logic Synthesis with Yosys

We now want to convert the RTL to a netlist. For this we will use an open-source tool called Yosys.

2.1 Tasks, Implementation Details

`synth.tcl` runs 7 Yosys steps: `read_verilog` → `synth -flatten` → `dfflibmap` → `abc -liberty` → `opt_clean -purge` → `stat` report → `write_verilog` netlist. Netlist checked against RTL via gate-level simulation (GLS) with SKY130 cell models. A testbench race (`#1` settle delay tying with gate `UNIT_DELAY`, causing stale reads) was found and fixed by using `@(negedge clk)` instead.

2.2 Deliverables

One of the deliverables was to simulate the synthesised netlist to prove that behaviour has not changed. The screenshot below shows that all the tests pass.

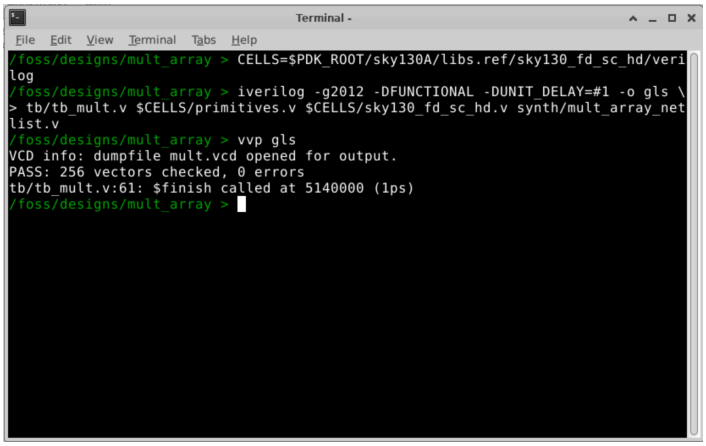


Figure 4. Netlist behaviour

These here are the results:

Metric	Value
Total number of cells	73
Number of flip-flops	16
Total cell area (μm^2)	805.77
Flip-flop area (μm^2)	400.38
Flip-flop area (% of total)	49.69%
3 most-used cell types	`sky130_fd_sc_hd__dfrtp_1` (16), `sky130_fd_sc_hd__nand2_1` (11), then a 4-way tie at 4 each: `and2_0`, `a21oi_1`, `xnor2_1`, `nand4_1`
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

Table 1. Synthesis and Simulation Metrics

3 Part-C: Static Timing Analysis with OpenSTA

3.1 Tasks, Implementation Details

`sta.tcl` runs 7 OpenSTA steps on `synth/mult_array_netlist.v` at the typical corner (`tt_025C_1v80`). Ran at $T = 10$ns to get the worst slack (WNS), computed $T_{\text{min}} = T - \text{WNS}$, then reran at T_{min} to confirm slack ≈ 0.

3.2 Deliverables

For Task-C, we have to report a table. This is given below:

Metric	Value
Worst slack at $T = 10\,\text{ns}$ (ns)	7.404
T_{min} (ns)	2.60
F_{max} (MHz)	384.62
Slack at $T = T_{\text{min}}$ (ns)	0.004
Critical path start point	`_118_` (rising-edge flip-flop Q, clocked by `clk`)
Critical path end point	`_112_` (rising-edge flip-flop D, clocked by `clk`)
Number of cells on the critical path	8
Latency (ns) $= 2 \times T_{\text{min}}$	5.20
Throughput (million results/sec)	192.31

Table 2. Static Timing Analysis Metrics

Critical path cells: in order (from `mult_array_timing.rpt`, the `clk`-group path — this is the actual worst path; the `asynchronous`/reset-recovery path group reported separately has a larger slack of 9.228ns at $T = 10$ and isn't the bottleneck):

1. `sky130_fd_sc_hd__and2_0`
2. `sky130_fd_sc_hd__or3_1`
3. `sky130_fd_sc_hd__and4_1`
4. `sky130_fd_sc_hd__o2111a_1`
5. `sky130_fd_sc_hd__o21ai_0`
6. `sky130_fd_sc_hd__xnor3_1`
7. `sky130_fd_sc_hd__maj3_1`
8. `sky130_fd_sc_hd__xor3_1`

4 Part-D: Physical Design

4.1 Tasks, Implementation Details

`config.json` sets `CLOCK_PERIOD` = $1.2 \times T_{\text{min}}$ from Part C, rounded up to 0.5ns (started at 3.5ns). Ran the LibreLane flow, increasing `CLOCK_PERIOD` by 0.5ns each time setup timing failed, until 0 DRC errors, 0 LVS errors and setup slack ≥ 0 – 5 attempts total, final `CLOCK_PERIOD` = 7.0ns. `metrics.json` and `mult_array.gds` copied from the passing run's `final/` folder.

4.2 Deliverables

Metric	Value
`CLOCK_PERIOD` used (ns)	7.0
Number of runs needed	5
Core area (μm^2)	17759.5
Cells after synthesis (Part B)	73
Cells after place and route	1896
Kinds of cells added	fill (1562), tap (245), clock buffers (3), timing-repair buffers (19), inverters (3)
Wirelength (μm)	2375
Setup slack (ns)	0.335
Post-route F_{max} (MHz)	150.04
F_{max} from Part C (MHz)	384.62
DRC errors	0
LVS errors	0

Table 3. Physical Design Metrics

Here are the screenshots of the layout (Figures 5 and 6):

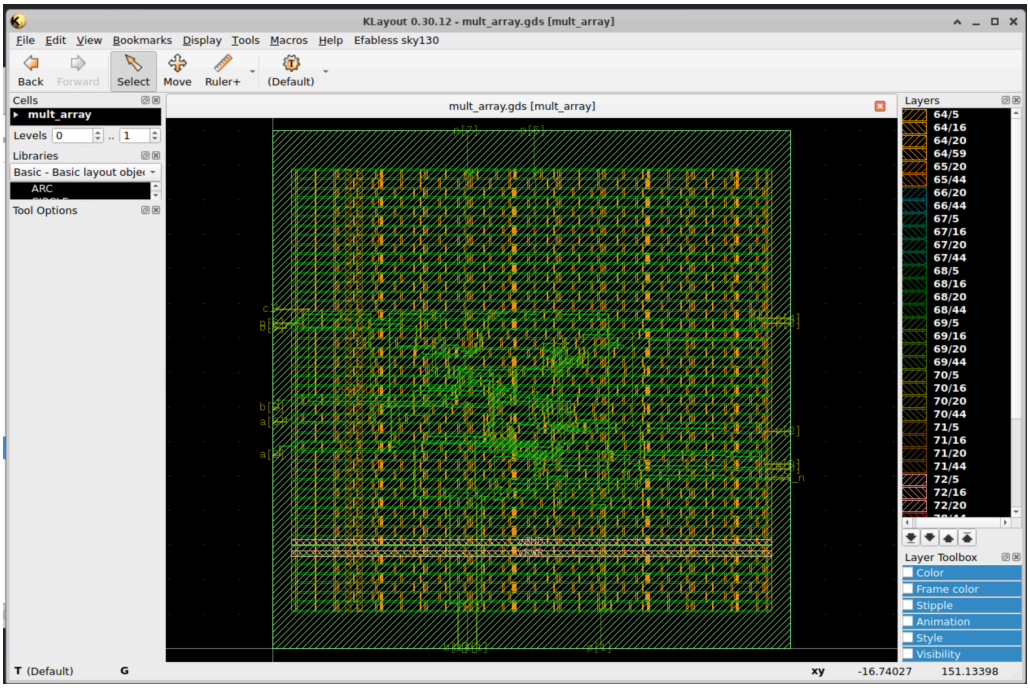


Figure 5. The whole die

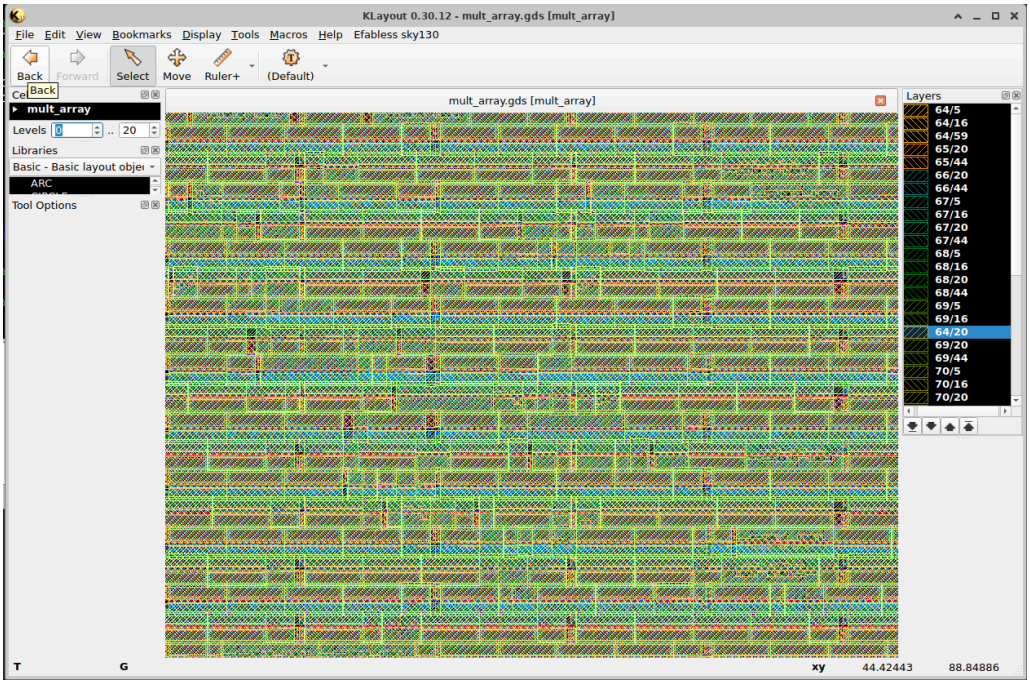


Figure 6. Zoomed in view

5 Questions and Answers

Q1. Synthesis produced 73 total cells, 16 of which are flip-flops, so 57 cells implement the logic – not $16 + 12 = 28$. ABC's technology mapping restructures the AND/full-adder Boolean logic across cell boundaries into a different set of optimized SKY130 cells (e.g. `nand2`, `a21oi`, `xnor2`), so the synthesized cell count has no 1:1 correspondence with the original RTL gate count.

Q2. The critical path runs from flip-flop `_118_` to flip-flop `_112_` (both internal pipeline registers), through 8 cells. This matches Section 2's estimate of about $2N = 8$ full-adder delays for $N = 4$ closely in stage count, though the 8 cells are ABC-optimized compound gates (`maj3`, `xor3`, etc.), not literal full-adder instances.

Q3. Flip-flops are 49.69% of total cell area. This is large because SKY130 `dfrtp` flip-flop cells are physically much bigger than simple combinational gates (master-slave latch + reset circuitry), and the design is tiny (73 cells total), so the fixed area cost of 16 bits of state dominates relative to the small amount of combinational logic.

Q4. Post-route $F_{\max}$ (150.04MHz) is lower than Part C's $F_{\max}$ (384.62MHz) because post-route sign-off uses the worst-case slow corner (`ss_100C_1v60`) with real wire RC parasitics, while Part C used an ideal, parasitic-free netlist at the typical corner only. The flow also added fill cells (legalize rows for DRC), tap cells (prevent latch-up), clock buffers (CTS, equalize clock arrival), and timing-repair buffers/inverters (resizer fixing setup violations) – all adding delay not present in the Part C netlist.

Q5. DRC (Design Rule Check) verifies the layout geometry obeys the foundry's manufacturing rules (minimum widths, spacings, enclosures), so the chip can be physically fabricated. LVS (Layout vs Schematic) verifies the layout's extracted netlist matches the intended circuit netlist, so the chip implements the correct design. Both must pass before manufacture: DRC alone doesn't guarantee correct connectivity, and LVS alone doesn't guarantee manufacturability – fabrication is far too costly to discover either failure after tape-out.

Q6. For $N = 8$: AND gates $= N^2 = 64$, full adders $= N(N-1) = 56$ (up from 12 for $N = 4$). Critical path $\approx 2N = 16$ full-adder delays (up from 8 for $N = 4$), i.e. the delay roughly doubles when N doubles, consistent with Section 2's "grows about linearly with N" statement.

Links, Team Contributions

GitHub repository: https://github.com/houleux/DVD-Asssignmet-1

Team Member	Contribution
Sai Rithvik Achutuni	Part-A, Part-C
Gautam Gandhi	Part-B, Part-D

Table 4. Team Contributions